

COMPTE RENDU

Ethical Hacking & Defense - TP2 - Phase de Scan : Maîtriser Nmap
4ème année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants : HATHOUTI Mohammed Taha
JIDAL Ilyas
Filière : Cybersecurité
Année : 2026/2027
Enseignant : M. SEBBAR & M. ZEROUAL
Date : 10 octobre 2026

Table des matières

Objectifs	2
Précautions Légales et Éthiques	2
1 Environnement de Travail	3
2 Scan réseau avec Nmap	4
2.1 Test 1 : Découverte des hôtes sur le réseau	4
2.1.1 Principe	4
2.1.2 Exécution et résultats	4
2.2 Test 2 : Scan TCP SYN	5
2.2.1 Principe du scan SYN (<i>half-open scan</i>)	5
2.2.2 Préparation : simulation des services sur la victime	6
2.2.3 Scan SYN par défaut	6
2.2.4 Scan ciblé sur les ports identifiés comme ouverts	7
2.2.5 Détection de version des services (-sV)	7
2.2.6 Scan d'un port fermé	8
3 OS Fingerprinting	9
3.1 Principe	9
3.2 Scan -O et -O -vv	9
3.3 Impact du pare-feu local (ufw)	10
3.4 Scan agressif (-A)	11
4 Évasion de pare-feu et IDS/IPS	12
4.1 Principe	12
4.2 Fragmentation basique	13
4.3 Scan SYN avec détection de version et leurres fixes	13
4.4 Scan SYN avec leurres aléatoires	14
4.5 Scan SYN, fragmentation et leurre combinés	14
4.6 Ajout des trames Ethernet brutes (--send-eth)	14
4.7 Fragmentation extrême (-mtu 16)	14
4.8 Synthèse et réponses aux questions de l'énoncé	15
Conclusion	16

Objectifs

L'objectif de ce TP est d'explorer des techniques avancées de scan réseau en maîtrisant **Nmap** : le moteur de scripts NSE, l'automatisation, les techniques d'évasion de pare-feu/IDS, et la fragmentation de paquets. Afin de bien comprendre le fonctionnement du processus de scan pour chaque type d'attaque, le trafic a été capturé et analysé avec **Wireshark**.

Ce TP est organisé en trois parties :

- **Partie 1** : Scan réseau avec Nmap ;
 - **Test 1** : Découverte des hôtes sur le réseau (*host discovery*) ;
 - **Test 2** : Scan TCP SYN et détection de version des services ;
- **Partie 3** : OS Fingerprinting (identification du système d'exploitation) ;
- **Partie 4** : Évasion de pare-feu et IDS/IPS par fragmentation et leurres.

Précautions Légales et Éthiques

IMPORTANT : Ce TP présente des techniques de base pouvant être utilisées pour commettre des actes illégaux. Il est jugé utile dans le cadre du cours pour mieux comprendre les types d'attaques pouvant être menées contre les systèmes que nous cherchons à protéger, mais ces tests n'ont jamais été appliqués à des systèmes réels sans autorisation : l'intégralité des scans de ce compte rendu a été réalisée sur un binôme de machines personnelles, isolées dans un réseau local privé, sans aucune cible tierce.

1 Environnement de Travail

Contrairement à l'énoncé original (rédigé pour un environnement avec machines virtuelles Windows 7 / Ubuntu / Metasploitable), nous avons réalisé ce TP sur **deux machines physiques sous Ubuntu**, reliées directement (sans Wi-Fi) par câbles RJ45 à une box Huawei EchoLife HG8245H5 utilisée en commutateur, sans accès à un réseau virtualisé :

- **Machine attaquante** : `hathouti@Taha-inspiron-16` (Dell Inspiron 16) — Ubuntu 24.04, interface `enx0c3796401539`, adresse **192.168.100.41** ;
- **Machine victime** : `hathouti@Taha-HP-ProBook` (HP ProBook) — Ubuntu, interface `enp0s31f6`, adresse **192.168.100.40** ;
- **Interconnexion** : box Huawei EchoLife HG8245H5 utilisée uniquement comme commutateur Ethernet local, les deux machines (attaquante et victime) y étant branchées directement par câbles RG45/RJ45 (aucune liaison Wi-Fi), sous-réseau 192.168.100.0/24, aucune passerelle utilisée (communication directe niveau 2) ;
- **Services simulés sur la victime** : écoutes `netcat` sur les ports 21 (FTP), 25 (SMTP) et 110 (POP3), en complément des vrais services **OpenSSH** (port 22) et **Apache2** (port 80) installés nativement.

```
hathouti@Taha-HP-ProBook:~/Bureau/UIR/4A/S7/Ethical-Hacking/TP2/Screen/Test1$ ifconfig
enp0s31f6: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.100.40 netmask 255.255.255.0 broadcast 192.168.100.255
    inet6 fe80::b400:5a53:3417:c1ba prefixlen 64 scopeid 0x20<link>
    ether b0:22:7a:da:13:07 txqueuelen 1000 (Ethernet)
    RX packets 263 bytes 29279 (29.2 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 335 bytes 31110 (31.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 16 memory 0x54100000-54120000
```

FIGURE 1 – Configuration réseau de la machine victime (HP ProBook, 192.168.100.40)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP2$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 82:d0:d4:c0:40:02 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 214 overruns 0 carrier 0 collisions 0

enx0c3796401539: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.100.41 netmask 255.255.255.0 broadcast 192.168.100.255
    inet6 fe80::e37:96ff:fe40:1539 prefixlen 64 scopeid 0x20<link>
    ether 0c:37:96:40:15:39 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

FIGURE 2 – Configuration réseau de la machine attaquante (Dell Inspiron-16, 192.168.100.41)

Note d'adaptation : n'ayant pas de machine Windows 7 ni de Metasploitable à disposition, toutes les cibles de l'énoncé ont été remplacées par la machine victime unique (192.168.100.40). Les ports FTP/SMTP/POP3, normalement fournis par des services réels sur Metasploitable, ont été simulés avec des écouteurs `netcat` (`nc -l -p <port>`) sur la victime. Cette adaptation est documentée section par section.

2 Scan réseau avec Nmap

2.1 Test 1 : Découverte des hôtes sur le réseau

2.1.1 Principe

Le *host discovery* (ou *ping scan*) permet d'identifier rapidement quelles adresses IP d'une plage sont actives, sans encore sonder leurs ports. Selon les privilèges et le type de réseau, Nmap combine plusieurs techniques : requêtes ARP (réseau local, nécessite les droits root), ICMP Echo, sondes TCP SYN/ACK vers des ports courants (443, 80), et requêtes ICMP Timestamp.

2.1.2 Exécution et résultats

Ping scan sur une plage ciblée

```
1 nmap -sP 192.168.100.35-45
```

Listing 1 – Sortie de `nmap -sP 192.168.100.35-45`

```
1 Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-10 11:03
   +00
2 Nmap scan report for 192.168.100.40
3 Host is up (0.00075s latency).
4 Nmap scan report for Taha-inspiron-16 (192.168.100.41)
5 Host is up (0.00025s latency).
6 Nmap done: 11 IP addresses (2 hosts up) scanned in 14.41 seconds
```

Sur les 11 adresses de la plage (.35 à .45), seules les deux adresses réellement occupées par nos machines répondent : la victime (.40) et l'attaquant lui-même (.41, détecté par son propre nom d'hôte Taha-inspiron-16).

Protocole utilisé : la capture Wireshark associée (45 paquets) montre un mélange d'ARP (29 paquets, résolution d'adresse MAC par le noyau Linux avant tout envoi) et de TCP (8 paquets, flags SYN/SYN-ACK/ACK/RST-ACK). Cette commande ayant été lancée **sans sudo**, Nmap n'a pas eu accès aux sockets bruts nécessaires à un balayage ARP natif; il est donc retombé sur une découverte par sondes **TCP SYN** vers les ports de service courants, ce que confirme la mention "**received syn-ack**" (et non "**received arp-response**") dans la sortie verbeuse ci-dessous. Les paquets ARP visibles dans la capture restent néanmoins présents : ils proviennent de la résolution d'adresse MAC effectuée par le noyau avant l'émission de chaque segment TCP, indépendamment de la technique de découverte choisie par Nmap.

Mode verbeux (-vv)

```
1 nmap -sP -vv 192.168.100.35-45
```

Listing 2 – Extrait de `nmap -sP -vv` – détail par hôte

```
1 Nmap scan report for 192.168.100.40
2 Host is up, received syn-ack (0.00072s latency).
3 Nmap scan report for Taha-inspiron-16 (192.168.100.41)
4 Host is up, received syn-ack (0.00026s latency).
5 Nmap done: 11 IP addresses (2 hosts up) scanned in 14.40 seconds
```

Le mode verbeux confirme, pour chaque hôte, la **raison exacte** de son état "up" (received syn-ack) ainsi que la latence précise. Les 9 adresses silencieuses sont explicitement marquées [host down, received no-response].

Balayage du sous-réseau complet (-sn -vv)

```
1 nmap -sn -vv 192.168.100.0/24
```

Listing 3 – Extrait de `nmap -sn -vv` sur le /24 complet (256 adresses)

```
1 Scanning 256 hosts [2 ports/host]
2 ...
3 Nmap scan report for _gateway (192.168.100.1)
4 Host is up, received syn-ack (0.00093s latency).
5 ...
6 Nmap scan report for 192.168.100.40
7 Host is up, received syn-ack (0.00065s latency).
8 Nmap scan report for Taha-inspiron-16 (192.168.100.41)
9 Host is up, received syn-ack (0.00057s latency).
10 ...
11 Nmap scan report for 192.168.100.255 [host down, received net-
    unreachable]
12 Nmap done: 256 IP addresses (3 hosts up) scanned in 15.71 seconds
```

Réponses aux questions de l'énoncé :

- *Quel protocole Nmap utilise-t-il avec l'option -sP pour détecter les appareils sur le réseau ?*
→ Par défaut sur un réseau local et avec les privilèges root, Nmap privilégie **ARP**. Dans notre cas, les commandes ayant été lancées sans `sudo`, Nmap est passé par une sonde **TCP SYN** vers le port 443 (comportement de repli documenté par Nmap pour l'hôte découverte non privilégiée). Le résultat final est identique (hôte détecté comme *up*), seule la méthode diffère ;
- *Combien d'hôtes ont répondu aux requêtes ICMP ?*
→ **Aucun**, puisqu'aucune requête ICMP Echo n'a été réellement utilisée ici : la découverte s'est appuyée sur des sondes TCP SYN, comme le montre la mention **received syn-ack** sur chacun des 3 hôtes détectés (.1, .40, .41) ;
- *Wireshark a-t-il capturé d'autres réponses protocolaires en dehors d'ICMP ?*
→ Oui : essentiellement des trames **ARP** (résolution d'adresse MAC par le noyau) et des segments **TCP** (SYN/SYN-ACK/RST-ACK), mais aucune trame ICMP, cohérent avec l'absence de sondes ICMP dans cette découverte non privilégiée. L'adresse de broadcast 192.168.100.255 a quant à elle généré une erreur ICMP **net-unreach** (réseau/hôte inaccessible), attendue pour une adresse de diffusion.

2.2 Test 2 : Scan TCP SYN

2.2.1 Principe du scan SYN (*half-open scan*)

Le scan SYN (-sS), dit **furtif** ou *half-open*, consiste à envoyer un paquet SYN vers le port cible sans jamais compléter la poignée de main TCP en trois temps :

- si le port est **ouvert**, la cible répond par **SYN/ACK** ; Nmap répond alors par un **RST** au lieu du **ACK** attendu, ce qui évite d'établir une connexion complète ;
- si le port est **fermé**, la cible répond directement par **RST/ACK** ;
- si le port est **filtré**, aucune réponse n'est reçue (ou un message ICMP de type "unreachable").

Avantages : plus rapide qu'un scan `connect()` classique (`-sT`) car il ne complète jamais la connexion ; plus discret, puisque la connexion n'est jamais pleinement établie et n'est donc généralement pas journalisée par l'application visée (seul le pare-feu/IDS au niveau noyau peut la voir).

Inconvénients : nécessite les **privileges root** pour forger des paquets bruts ; reste détectable par un IDS qui surveille les paquets SYN sans ACK consécutif ; moins fiable qu'un scan `connect()` sur des cibles au comportement TCP non standard.

2.2.2 Préparation : simulation des services sur la victime

```
1 nc -l -p 21 & nc -l -p 53 & nc -l -p 110 & nc -l -p 25
```

Listing 4 – Sortie obtenue sur la victime

```
1 nc: Address already in use
2 Listening on 0.0.0.0 25
3 Listening on 0.0.0.0 21
4 Listening on 0.0.0.0 110
```

Trois écoutes démarrent correctement (21, 25, 110) ; le port **53** échoue avec **Address already in use**, un service système (probablement **systemd-resolved**) occupant déjà ce port sur la victime. Par ailleurs, la commande telle que tapée ne place pas de **&** après le dernier `nc -l -p 25` : celui-ci tourne donc au **premier plan**, contrairement aux trois autres lancés en arrière-plan. Cette nuance a eu une conséquence observée plus bas (port 25 fermé prématurément).

2.2.3 Scan SYN par défaut

```
1 sudo nmap -sS 192.168.100.40
```

Listing 5 – Sortie de `nmap -sS` (ports par défaut)

```
1 Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-10 12:09
   +00
2 Nmap scan report for 192.168.100.40
3 Host is up (0.00040s latency).
4 Not shown: 995 closed tcp ports (reset)
5 PORT      STATE SERVICE
6 21/tcp    open  ftp
7 22/tcp    open  ssh
8 25/tcp    open  smtp
9 80/tcp    open  http
10 110/tcp   open  pop3
11 MAC Address: B0:22:7A:DA:13:07 (HP)
12 Nmap done: 1 IP address (1 host up) scanned in 13.20 seconds
```

Les 5 ports attendus sont bien détectés **ouverts** : les trois écoutes **nc** (21/25/110) ainsi que les deux vrais services (22/SSH, 80/HTTP).

Analyse de la capture Wireshark associée (2021 paquets côté attaquant, 2034 côté victime) : le filtre `tcp.flags.syn==1 && tcp.flags.ack==0` isole les 1000 paquets SYN envoyés par Nmap (un par port par défaut scanné). Les réponses se répartissent en **5 paquets** SYN/ACK (les 5 ports ouverts) et **995 paquets** RST/ACK (les ports fermés), confirmant exactement la sortie texte de Nmap (Not shown: 995 closed tcp ports).

2.2.4 Scan ciblé sur les ports identifiés comme ouverts

Après avoir identifié les 5 ports ouverts, nous avons regroupé leur vérification en une seule commande plutôt que de les tester un par un :

```
1 sudo nmap -sS -p 21,22,25,80,110 192.168.100.40
```

Listing 6 – Sortie de `nmap -sS -p 21,22,25,80,110`

```
1 Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-10 12:18
   +00
2 Nmap scan report for 192.168.100.40
3 Host is up (0.0016s latency).
4
5 PORT      STATE SERVICE
6 21/tcp    open  ftp
7 22/tcp    open  ssh
8 25/tcp    closed smtp
9 80/tcp    open  http
10 110/tcp   open  pop3
11 MAC Address: B0:22:7A:DA:13:07 (HP)
12 Nmap done: 1 IP address (1 host up) scanned in 13.16 seconds
```

Observation inattendue : le port **25** apparaît maintenant **fermé**, alors qu'il était ouvert 9 minutes plus tôt. L'explication tient à la préparation de la section précédente : le processus `nc -l -p 25` tournait au **premier plan** (absence de `&` terminal après cette commande dans la chaîne). Entre les deux scans, une annulation manuelle (`Ctrl+C`) du terminal a tué uniquement ce processus au premier plan, laissant les écoutes 21 et 110 (lancées en arrière-plan) actives. C'est un bon exemple pratique de la fragilité de **netcat** sans l'option `-k` (*keep-listening*) : sans elle, un écouteur ne survit de toute façon qu'à une seule connexion acceptée, et ici il a en plus été interrompu manuellement.

2.2.5 Détection de version des services (`-sV`)

```
1 sudo nmap -sV 192.168.100.40
```

Listing 7 – Sortie de `nmap -sV`

```
1 Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-10 13:46
   +00
2 Nmap scan report for 192.168.100.40
3 Host is up (0.00029s latency).
4 Not shown: 998 closed tcp ports (reset)
```

```

5  PORT      STATE SERVICE VERSION
6  22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu
      Linux; protocol 2.0)
7  80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
8  MAC Address: B0:22:7A:DA:13:07 (HP)
9  Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
10
11 Nmap done: 1 IP address (1 host up) scanned in 19.45 seconds

```

Seuls **22** et **80** sont désormais détectés ouverts : entre-temps, les écouteurs `nc` simulant FTP/SMTP/POP3 avaient été fermés (ils ne survivent pas durablement sans `-k`, voir ci-dessus). Pour les deux ports réels en revanche, `-sV` va bien au-delà d'un simple scan SYN : il **complète la connexion TCP** et dialogue avec le service pour récupérer sa bannière, révélant **OpenSSH 9.6p1 (Ubuntu)** et **Apache httpd 2.4.58 (Ubuntu)**, ainsi que le système d'exploitation sous-jacent (**Linux**) directement déduit de la bannière SSH.

Analyse de la capture Wireshark associée (filtre `tcp.port==22` ou `tcp.port==80`) : contrairement au scan `-sS` pur, on observe ici des échanges TCP **complets** (SYN, SYN/ACK, ACK, puis des segments PSH/ACK contenant la bannière du service en clair, et enfin FIN/ACK), la preuve que `-sV` termine la poignée de main pour chaque port ouvert, à la différence du scan furtif qui s'arrête au RST.

2.2.6 Scan d'un port fermé

```

1  sudo nmap -sS 192.168.100.40 -p 23

```

Listing 8 – Sortie de `nmap -sS -p 23` (port fermé)

```

1  Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-10 14:04
      +00
2  Nmap scan report for 192.168.100.40
3  Host is up (0.00087s latency).
4
5  PORT      STATE SERVICE
6  23/tcp    closed telnet
7  MAC Address: B0:22:7A:DA:13:07 (HP)
8  Nmap done: 1 IP address (1 host up) scanned in 13.13 seconds

```

Comparaison port ouvert / port fermé dans Wireshark :

Critère	Port ouvert (ex. 80)	Port fermé (23)
Paquets échangés	SYN → SYN/ACK → RST	SYN → RST/ACK
Nombre de paquets TCP (capture Q7)	–	2 (1 S, 1 RA)
Qui ferme l'échange ?	L'attaquant (Nmap, via RST)	La victime (directement)
État annoncé par Nmap	open	closed

TABLE 1 – Différence de comportement TCP entre un port ouvert et un port fermé

La capture associée au port fermé (23) ne contient que **2 paquets TCP** (le SYN émis, le RST/ACK reçu), contre 3 étapes pour un port ouvert avant la fermeture forcée par Nmap.

C'est précisément cette économie d'un aller-retour qui rend le scan SYN rapide et discret sur un grand nombre de ports fermés.

3 OS Fingerprinting

3.1 Principe

L'OS fingerprinting (-O) consiste à envoyer une série de sondes TCP/UDP/ICMP volontairement atypiques, puis à comparer les particularités de la pile TCP/IP de la cible (taille de fenêtre initiale, options TCP, génération des numéros de séquence, TTL initial, réponses ICMP) à une base de signatures. Chaque système d'exploitation implémente ces détails différemment, ce qui permet de déduire l'OS (et parfois la version du noyau) sans jamais s'y connecter.

3.2 Scan -O et -O -vv

```
1 sudo nmap -O 192.168.100.40
2 sudo nmap -O -vv 192.168.100.40
```

Listing 9 – Sortie de nmap -O

```
1 Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-10 14:23
   +00
2 Nmap scan report for 192.168.100.40
3 Host is up (0.00076s latency).
4 Not shown: 998 closed tcp ports (reset)
5 PORT      STATE SERVICE
6 22/tcp    open  ssh
7 80/tcp    open  http
8 MAC Address: B0:22:7A:DA:13:07 (HP)
9 Device type: general purpose
10 Running: Linux 5.X
11 OS CPE: cpe:/o:linux:linux_kernel:5
12 OS details: Linux 5.3 - 5.4
13 Network Distance: 1 hop
14
15 Nmap done: 1 IP address (1 host up) scanned in 14.47 seconds
```

Nmap identifie correctement la famille **Linux**, mais estime un noyau **5.3 – 5.4**. Comme les deux machines sont sous Ubuntu avec un noyau nettement plus récent (le scan -sV a révélé OpenSSH 9.6p1, typique d'Ubuntu 24.04 et d'un noyau 6.x), cette estimation illustre une limite réelle du fingerprinting : la base de signatures de Nmap n'est pas mise à jour à la même vitesse que les noyaux Linux, et les piles TCP/IP récentes évoluent peu entre versions majeures, ce qui brouille la précision de l'estimation.

Le mode -vv confirme que la découverte d'hôte a cette fois utilisé **ARP** ("received arp-response", contrairement au Test 1), car -O nécessite les privilèges root pour forger ses sondes, et Nmap bascule alors automatiquement sur ARP pour la découverte locale. L'empreinte brute (TCP/IP fingerprint) est également affichée en détail : taille de fenêtre (W=FE88), options TCP (M5B4ST11NWA), TTL de 64 (affiché comme T=40 en hexadécimal), génération des ISN (SP=108, GCD=1), etc.

3.3 Impact du pare-feu local (ufw)

N'ayant pas de pare-feu Windows à disposition, nous avons reproduit ce test avec ufw (pare-feu natif d'Ubuntu) activé puis désactivé sur la victime.

```
hathouti@Taha-HP-ProBook:~/Bureau/UIR/4A/S7/Ethical-Hacking/TP2/Scan_Reseau_NMAP/test2$ sudo ufw enable
[sudo] Mot de passe de hathouti :
Le pare-feu est actif et lancé au démarrage du système
```

FIGURE 3 – Activation d'ufw sur la victime

Listing 10 – nmap -O avec ufw actif

```
1 Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-10 14:26
  +00
2 Nmap scan report for 192.168.100.40
3 Host is up (0.00092s latency).
4 All 1000 scanned ports on 192.168.100.40 are in ignored states.
5 Not shown: 1000 filtered tcp ports (no-response)
6 MAC Address: B0:22:7A:DA:13:07 (HP)
7 Too many fingerprints match this host to give specific OS details
8 Network Distance: 1 hop
9
10 Nmap done: 1 IP address (1 host up) scanned in 36.45 seconds
```

```
hathouti@Taha-HP-ProBook:~/Bureau/UIR/4A/S7/Ethical-Hacking/TP2/Scan_Reseau_NMAP/test2$ sudo ufw disable
Le pare-feu est arrêté et désactivé lors du démarrage du système
hathouti@Taha-HP-ProBook:~/Bureau/UIR/4A/S7/Ethical-Hacking/TP2/Scan_Reseau_NMAP/test2$
```

FIGURE 4 – Désactivation d'ufw sur la victime

Listing 11 – nmap -O avec ufw désactivé

```
1 Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-10 14:27
  +00
2 Nmap scan report for 192.168.100.40
3 Host is up (0.0011s latency).
4 Not shown: 998 closed tcp ports (reset)
5 PORT      STATE SERVICE
6 22/tcp    open  ssh
7 80/tcp    open  http
8 MAC Address: B0:22:7A:DA:13:07 (HP)
9 No exact OS matches for host (If you know what OS is running on
  it, see https://nmap.org/submit/ ).
10 ...
11 Nmap done: 1 IP address (1 host up) scanned in 24.45 seconds
```

Différences observées :

	ufw actif	ufw inactif
Ports détectés	filtered (1000/1000)	open (22, 80), closed (998)
Durée du scan	36,45 s (plus lent)	24,45 s
Résultat OS	<i>Too many fingerprints match</i>	<i>No exact OS matches</i>

TABLE 2 – Effet d’ufw sur la détection de ports et le fingerprinting

Lorsqu’ufw est actif, la politique par défaut bloque silencieusement les paquets entrants non sollicités : Nmap ne reçoit ni SYN/ACK, ni RST, et classe donc tous les ports comme **filtered**. Sans aucun port ouvert exploitable pour l’empreinte TCP active, Nmap ne peut comparer que les réponses passives (ou absentes), d’où un résultat beaucoup plus incertain (*"too many fingerprints match"*) et un scan nettement plus lent (chaque sonde sans réponse doit être retransmise avant d’être déclarée filtrée).

Fait intéressant : même ufw désactivé, le résultat diffère légèrement du tout premier scan -O (*"Linux 5.3 - 5.4"* contre *"No exact OS matches"* ici). La comparaison des deux empreintes brutes montre un léger écart sur le paramètre SP (108 contre 109), qui mesure la difficulté de prédiction des numéros de séquence TCP : cette valeur varie légèrement à chaque exécution car elle dépend d’un calcul statistique sur plusieurs sondes successives, illustrant que le fingerprinting OS n’est **jamais parfaitement déterministe**, même sur une cible qui n’a pas changé entre deux scans consécutifs.

3.4 Scan agressif (-A)

```
1 sudo nmap -A 192.168.100.40
```

Listing 12 – Sortie de nmap -A

```
1 Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-10 14:50
   +00
2 Nmap scan report for 192.168.100.40
3 Host is up (0.00082s latency).
4 Not shown: 998 closed tcp ports (reset)
5 PORT      STATE SERVICE VERSION
6 22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu
   Linux; protocol 2.0)
7 | ssh-hostkey:
8 |   256 b5:48:1a:2d:5e:ca:c0:e2:d3:8a:7c:fc:47:55:e0:d0 (ECDSA)
9 |_  256 dd:6b:7c:13:3f:4a:c8:39:3c:9d:b0:78:de:df:7f:9e (ED25519)
10 80/tcp    open  http      Apache httpd 2.4.58 ((Ubuntu))
11 |_http-title: Apache2 Ubuntu Default Page: It works
12 |_http-server-header: Apache/2.4.58 (Ubuntu)
13 MAC Address: B0:22:7A:DA:13:07 (HP)
14 Device type: general purpose
15 Running: Linux 5.X
16 OS details: Linux 5.3 - 5.4
17 Network Distance: 1 hop
18
19 TRACEROUTE
```

20	HOP	RTT	ADDRESS
21	1	0.82 ms	192.168.100.40
22			
23	Nmap done: 1 IP address (1 host up) scanned in 21.07 seconds		

Analyse de la capture Wireshark (2519 paquets côté attaquant, 2492 côté victime) :

- le filtre `tcp` isole la grande majorité des paquets (2475), qui regroupent la détection de ports (1000 `SYN`), l'OS fingerprinting, et la détection de version/scripts NSE sur les 2 ports ouverts (poignées de main complètes `SYN/SYN-ACK/ACK`, données applicatives en `PSH-ACK` pour récupérer les bannières SSH et HTTP, puis clôtures `FIN-ACK`) ;
- le filtre `icmp` révèle 6 paquets : ils proviennent du **traceroute intégré** au scan agressif, ici réduit à un unique saut (1 0.82 ms 192.168.100.40) puisque les deux machines sont directement reliées par le commutateur, sans routeur intermédiaire ;
- le filtre `udp` (32 paquets) correspond essentiellement au bruit de fond du réseau local (requêtes mDNS/SSDP), sans rapport direct avec le scan Nmap lui-même.

Réponses aux questions de l'énoncé :

- *Quelles informations supplémentaires ont été recueillies par le scan agressif ?*
→ Par rapport à un simple `-sS`, le scan `-A` combine **quatre** analyses en une seule commande : détection de version des services (`-sV`), détection d'OS (`-O`), exécution de scripts NSE par défaut (`-sC`, ici les clés hôtes SSH et le titre de la page Apache) et un **traceroute**. C'est la combinaison la plus riche en information en une seule commande ;
- *Comment les différents types de paquets (ICMP, TCP, UDP) sont-ils apparus dans Wireshark ?*
→ Le TCP domine très largement (scan de ports + interactions applicatives), l'ICMP n'apparaît qu'en petite quantité et sert exclusivement au traceroute, et l'UDP capturé est indépendant du scan (trafic de découverte réseau ambiant : mDNS, SSDP).

4 Évasion de pare-feu et IDS/IPS

4.1 Principe

L'objectif de cette partie est d'observer comment la **fragmentation de paquets** et l'usage de **leurres** (*decoys*) modifient la signature d'un scan Nmap, telle qu'elle apparaît dans une capture réseau. N'ayant pas d'IDS/IPS actif sur notre réseau, l'analyse porte principalement sur le **mécanisme** de ces techniques (comment les fragments sont construits et réassemblés, comment les leurres apparaissent comme sources alternatives) plutôt que sur une évasion démontrée contre un équipement de sécurité réel.

Adaptation : l'énoncé cible `nmap.scanme.org`, inaccessible dans notre réseau local isolé lors de la réalisation du TP ; toutes les commandes ont donc été redirigées vers la victime locale 192.168.100.40, et les leurres choisis dans notre propre sous-réseau (192.168.100.50 et .51, adresses inoccupées) pour rester analysables dans Wireshark.

4.2 Fragmentation basique

```
1 sudo nmap -f 192.168.100.40
```

Listing 13 – Sortie de `nmap -f`

```
1 Nmap scan report for 192.168.100.40
2 Host is up (0.00035s latency).
3 Not shown: 998 closed tcp ports (reset)
4 PORT      STATE SERVICE
5 22/tcp    open  ssh
6 80/tcp    open  http
7 MAC Address: B0:22:7A:DA:13:07 (HP)
8 Nmap done: 1 IP address (1 host up) scanned in 13.20 seconds
```

Le résultat (ports ouverts) est strictement identique à un scan `-sS` classique : la fragmentation ne change rien à l'issue du scan sur un réseau local sans IDS, seule la **forme** des paquets sur le réseau change.

Analyse Wireshark (filtre `ip.frag_offset > 0`) : sur 4047 paquets capturés côté attaquant, **3000 sont des fragments IP** (offset > 0 ou drapeau *More Fragments* positionné), contre seulement 2 paquets SYN/ACK et 998 RST/ACK au niveau TCP. Chaque segment TCP de 20 octets (en-tête minimal) que Nmap voulait envoyer en un seul paquet a été **scindé en plusieurs fragments IP de 8 octets de charge utile chacun** (taille par défaut de l'option `-f`), ce qui multiplie mécaniquement le nombre de paquets observés sur le réseau. Côté victime, le noyau Linux **réassemble silencieusement** ces fragments avant de les transmettre à la pile TCP : Wireshark affiche la mention *"Reassembled IP"* dans les détails du dernier fragment de chaque groupe, confirmant que la victime reconstitue correctement le segment original malgré sa fragmentation.

4.3 Scan SYN avec détection de version et leurres fixes

```
1 sudo nmap -sS -sV -F -D 192.168.100.50,192.168.100.51
   192.168.100.40
```

Listing 14 – Sortie de `nmap -sS -sV -F -D <leurres>`

```
1 Nmap scan report for 192.168.100.40
2 Host is up (0.00045s latency).
3 Not shown: 98 closed tcp ports (reset)
4 PORT      STATE SERVICE VERSION
5 22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu
   Linux; protocol 2.0)
6 80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
7 MAC Address: B0:22:7A:DA:13:07 (HP)
8 Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
9
10 Nmap done: 1 IP address (1 host up) scanned in 19.35 seconds
```

Analyse Wireshark : sur 562 paquets côté attaquant, le décompte des adresses sources montre trois origines bien distinctes et quasiment équilibrées : **192.168.100.41** (178 paquets, l'attaquant réel), **192.168.100.50** et **192.168.100.51** (100 paquets chacun, les

deux leurres). Pour un observateur qui n'examinerait que les adresses IP sources des sondes SYN sans disposer d'informations de niveau liaison (adresse MAC), les trois adresses apparaissent comme des scanners potentiels concurrents, ce qui dilue l'identification du véritable attaquant. En pratique, sur ce réseau local, l'adresse MAC source de **tous** les paquets (y compris ceux portant une IP leurre) reste celle de la carte réseau du Dell, ce qui trahirait immédiatement la supercherie à quiconque inspecterait la couche 2 – une limite importante des leurres sur un segment Ethernet partagé.

4.4 Scan SYN avec leurres aléatoires

```
1 sudo nmap -sS -sV -F -D RND:3 192.168.100.40
```

Résultat identique aux ports ouverts (22, 80), mais cette fois Nmap génère lui-même 3 adresses IP aléatoires. La capture confirme ce tirage avec des adresses **totale­ment hors de notre sous-réseau** : 218.230.231.175, 59.22.212.236 et 27.195.59.59 (100 paquets chacun), aux côtés de l'attaquant réel (192.168.100.41, 165 paquets) et de la victime. Contrairement aux leurres fixes du test précédent, ces adresses aléatoires n'appartiennent à aucun réseau local plausible : sur un vrai Internet, un analyste expérimenté pourrait les écarter rapidement du fait de leur caractère disparate (plages géographiques incohérentes avec le contexte), ce qui illustre une limite connue de l'option RND.

4.5 Scan SYN, fragmentation et leurre combinés

```
1 sudo nmap -sS -sV -F -f -D 192.168.100.50 192.168.100.40
```

Résultat inchangé (22, 80 ouverts). Capture : 831 paquets côté attaquant, dont **600 fragments IP** (contre 200 segments TCP complets), et deux sources bien visibles : 192.168.100.41 (369 paquets) et le leurre 192.168.100.50 (300 paquets, soit exactement moitié des 600 fragments). Cela confirme que **chaque sonde SYN, y compris celles portant l'adresse leurre, est fragmentée de la même façon** : la fragmentation et l'usurpation d'adresse source sont deux mécanismes indépendants qui se combinent sans interférer l'un avec l'autre.

4.6 Ajout des trames Ethernet brutes (--send-eth)

```
1 sudo nmap -sS -sV -F -f --send-eth -D 192.168.100.50
  192.168.100.40
```

Résultat identique. L'option **--send-eth** force Nmap à construire lui-même les trames au niveau **liaison** (couche 2) plutôt que de laisser le noyau s'en charger via sa pile IP habituelle. Dans notre cas, sur un simple commutateur Ethernet sans particularité, cela ne change rien de visible dans Wireshark par rapport au test précédent (828 paquets, 600 fragments, mêmes sources) : l'intérêt de cette option se manifeste surtout sur des configurations réseau particulières (interfaces virtuelles, tunnels, certains pare-feu bas niveau) où l'envoi via la pile IP standard du noyau poserait problème.

4.7 Fragmentation extrême (-mtu 16)

```
1 sudo nmap -sS -sV -F -f -mtu 16 --send-eth -D 192.168.100.50
192.168.100.40
```

Résultat toujours identique (22, 80 ouverts), mais la capture change radicalement d'échelle : **652 paquets** au total pour seulement 100 ports scannés, avec **400 fragments IP**. En forçant une MTU de 16 octets (soit 8 octets de charge utile IP par fragment après retrait de l'en-tête de 20 octets réajusté à un multiple de 8), chaque paquet TCP se retrouve découpé en un nombre de fragments bien plus élevé que pour la fragmentation par défaut (8 octets de charge utile déjà, donc le gain est ici surtout dans la constance du découpage extrême même pour les en-têtes TCP normalement non fragmentés). Ce test illustre la limite pratique de la fragmentation : au-delà d'un certain point, elle n'apporte plus de discrétion supplémentaire mais ralentit et complique sensiblement le scan, sans changer l'issue sur une cible qui réassemble correctement les fragments.

4.8 Synthèse et réponses aux questions de l'énoncé

Commande	Paquets totaux	Fragments IP	Sources IP distinctes
-f	4047	3000	1 (attaquant)
-D (leurre fixe)	562	0	3 (attaquant + 2 leures)
-D RND:3	630	0	4 (attaquant + 3 leures aléatoires)
-f -D (1 leurre)	831	600	2 (attaquant + 1 leurre)
-f --send-eth -D	828	600	2 (attaquant + 1 leurre)
-f -mtu 16 --send-eth -D	652	400	2 (attaquant + 1 leurre)

TABLE 3 – Synthèse quantitative des six scans d'évasion (capture côté attaquant)

Les paquets fragmentés ont-ils permis de contourner le pare-feu ou l'IDS ?

→ Notre réseau local ne disposant d'aucun IDS/IPS actif ni d'`ufw` configuré pendant ces tests, aucune évasion n'a été réellement nécessaire ni démontrée : le résultat du scan (ports ouverts/fermés) reste rigoureusement identique avec ou sans fragmentation ni leurre. Ce que ces tests démontrent en revanche, c'est le **mécanisme** : la fragmentation répartit un même paquet logique en plusieurs paquets physiques plus petits, ce qui peut mettre en échec un pare-feu ou un IDS **basique** qui n'inspecte chaque paquet qu'individuellement sans réassemblage préalable (il ne verrait jamais, par exemple, l'en-tête TCP complet dans un seul paquet). Un pare-feu qui filtre uniquement par **adresse IP source** (comme une règle `ufw deny from <IP>`) ne serait en revanche pas trompé par la fragmentation : il verrait toujours l'adresse IP réelle de l'attaquant dans chaque fragment (seule l'adresse du **leurre** changerait cela, en multipliant les adresses sources apparentes).

Comment les paquets fragmentés ont-ils été traités et réassemblés par la cible, d'après Wireshark ?

→ Dans toutes nos captures côté victime, les fragments successifs d'un même paquet (identifiés par le même champ *Identification* de l'en-tête IP) ont été **réassemblés sans délai ni erreur visible** par le noyau Linux de la victime : chaque groupe de fragments aboutit à un paquet TCP valide traité normalement par la pile réseau (réponse **SYN/ACK** ou **RST/ACK** identique à un scan non fragmenté). Wireshark matérialise ce réassemblage

en regroupant visuellement les fragments sous un paquet final marqué "*Reassembled IP*" dans son panneau de détails. Aucun problème de réassemblage (fragments manquants, délais anormaux, timeouts) n'a été observé, ce qui est cohérent avec un réseau local fiable et une pile IP standard et à jour sur la victime.

Conclusion

Ce TP a permis de mettre en pratique l'ensemble des grandes familles de scan de Nmap : découverte d'hôtes, scan de ports TCP SYN, détection de version et d'OS, et enfin techniques d'évasion par fragmentation et leurres. L'absence de machines Windows/Metasploitable nous a conduits à adapter systématiquement les cibles de l'énoncé à notre propre binôme de machines Ubuntu reliées en local, et à simuler certains services (FTP, SMTP, POP3) avec **netcat** – une contrainte qui a elle-même été une source d'observations intéressantes (fragilité d'un écouteur **nc** sans **-k**, différences de méthode de découverte d'hôte selon les privilèges). L'analyse croisée systématique des sorties Nmap et des captures Wireshark correspondantes a permis de confirmer, paquet par paquet, les mécanismes théoriques vus en cours : la nature *half-open* du scan SYN, le réassemblage transparent de la fragmentation IP, et les limites concrètes du fingerprinting OS et des leurres sur un réseau local sans équipement de filtrage actif.